Лабораторна робота №4

Тема: “Dining Philosophers” (Задача про обідаючих філософів)

Дисципліна: Розподілені системи

Виконав: *Божевський Іван Петрович*

Група: *ІНФ-1*

2) Мета роботи

Дослідити засоби синхронізації конкурентних процесів на прикладі класичної задачі Dining Philosophers, реалізувати два коректні підходи для `N=5` філософів у C# (.NET), перевірити відсутність deadlock та практичну відсутність starvation за обмежений час симуляції.

3) Постановка задачі

Є `5` філософів і `5` виделок.

Кожен філософ циклічно переходить через стани:

- `THINKING`
- `HUNGRY`
- `EATING`
- `PUT_FORKS`

Для переходу в `EATING` філософ повинен одночасно мати **дві** виделки: ліву і праву.

Вимоги:

- забезпечити взаємне виключення для виделок;
- уникнути взаємного блокування (**deadlock**);
- бажано уникати голодування (**starvation**) у практичному запуску.

4) Теоретичні відомості

4.1) Конкурентність, взаємне виключення, семафор, м'ютекс

- **Конкурентність**: одночасне виконання кількох незалежних задач.
- **Взаємне виключення**: гарантія, що критичний ресурс використовується лише одним потоком/задачею в момент часу.
- **Семафор**: лічильник дозволів доступу; для `SemaphoreSlim(1,1)` поводиться як бінарний замок.
- **М'ютекс (mutex/lock)**: механізм, що дозволяє лише одному виконавцю увійти в критичну секцію.

4.2) Deadlock: умови виникнення і як їх ламають

Deadlock зазвичай пов'язаний з умовами взаємного виключення, утримання і очікування, відсутності примусового звільнення та циклічного очікування.

У цій роботі циклічне очікування руйнується:

- у рішенні 1 — обмеженням числа одночасних претендентів (`room=N-1`);
- у рішенні 2 — серіалізацією захоплення пари виделок через глобальний м'ютекс.

4.3) Starvation (голодування)

Starvation — ситуація, коли окремий учасник дуже довго (або нескінченно) не отримує ресурс.

Це важливо, бо система може бути без deadlock, але несправедливою для окремих задач.

5) Опис алгоритмів

5.1) Рішення 1: один лічильний семафор `room = N-1`

Ідея: не дозволяти всім 5 філософам одночасно намагатися брати виделки.

Працює глобальний семафор `room` з початковим значенням `4`.

Послідовність:

1. Філософ стає `HUNGRY`.
2. `await room.WaitAsync(ct)`.

3. Захоплює ліву і праву виделки (синхронізовані ресурси).
4. Переходить у `EATING`.
5. Кладе виделки (`PUT_FORKS`).
6. `room.Release()`.

Чому немає дедлоку: принаймні один філософ завжди поза “кімнатою”, отже повний цикл взаємного очікування всіх 5 учасників неможливий.

Псевдокод (рішення 1):

```
room = Semaphore(N-1)
fork[0..4] = resources (lock або SemaphoreSlim(1,1))

loop until cancelled:
  THINKING (random 100..500 ms)
  HUNGRY
  wait(room)
  acquire(fork[left])
  acquire(fork[right])
  EATING (random 100..500 ms)
  release(fork[right])
  release(fork[left])
  PUT_FORKS
  release(room)
```

5.2) Рішення 2: один м’ютекс + п’ять семафорів для виделок

Ідея: використати глобальний м’ютекс для впорядкованого захоплення **двох** виделок як атомарної дії з точки зору конкуренції між філософами.

Послідовність:

1. Філософ стає `HUNGRY`.
2. Входить у `lock(globalMutex)`.
3. Всередині lock виконує:
   - `await/Wait forkSem[left]`
   - `await/Wait forkSem[right]`
4. Виходить із lock, їсть.

5. Після їжі звільняє виделки.

Чому немає дедлоку: одночасне “збирання” пари виделок серіалізоване, тому циклічного очікування між кількома філософами на етапі захоплення не утворюється.

Псевдокод (рішення 2):

```
globalMutex = lock object
forkSem[0..4] = SemaphoreSlim(1,1)

loop until cancelled:
  THINKING (random 100..500 ms)
  HUNGRY
  lock(globalMutex):
    wait(forkSem[left])
    wait(forkSem[right])
  EATING (random 100..500 ms)
  release(forkSem[right])
  release(forkSem[left])
  PUT_FORKS
```

6) Реалізація

6.1) Загальна структура програми

- Мова/платформа: **C# .NET**, консольний застосунок.
- На старті показується меню:
  - `1) N=5: one counting semaphore (room=N-1) + forks`
  - `2) N=5: one mutex + five semaphores for forks`
  - `3) Exit`
- Після вибору 1/2 вводяться:
  - тривалість симуляції в секундах (`default 10`);
  - `seed` для генератора (`default 123`).

6.2) Модель виделок і філософів

Для `N=5`:

- `leftFork = i`
- `rightFork = (i + 1) % N`

Вилка `i` розташована між філософами `i` і `(i+1)%N`.

6.3) Використання Task + CancellationToken

- Кожен філософ запускається в окремому `Task`.
- Симуляція завершується через `CancellationTokenSource` після заданого часу.
- Затримки станів реалізовано через `Task.Delay(..., ct)`.
- При скасуванні коректно обробляється `OperationCanceledException`, задачі завершуються штатно.

6.4) Логування та підрахунок `eatCount`

Логується кожна значуща дія:

- `timestamp`
- `philosopherId`
- `state` (`THINKING/HUNGRY/EATING/PUT_FORKS`)

Ведеться масив статистики `eatCount[i]` — кількість прийомів їжі кожним філософом.

6.5) Самоперевірка

Після завершення симуляції виконується self-check:

- якщо для будь-якого `i`: `eatCount[i] == 0`, виводиться `WARNING`;
- інакше виводиться успішний результат перевірки.

7) Результати експериментів

Таблиця результатів

Mode	Seconds	EatCount (0..4)	MinEatCount	
1) Semaphore room (`N-1`)	10	[5, 5, 6, 6, 6]	5	

Mode	Seconds	EatCount (0..4)	MinEatCount	
2) Mutex + fork semaphores	10	[6, 7, 6, 7, 7]	6	

Приклад логів

```
[22:39:05.374] P2 THINKING
[22:39:05.380] P3 THINKING
[22:39:05.568] P4 HUNGRY
[22:39:05.569] P4 EATING
[22:39:06.031] P4 PUT_FORKS
[22:39:06.031] P3 EATING
[22:39:06.699] P2 PUT_FORKS
[22:39:06.699] P1 EATING
```

Короткий аналіз спостережень

За обома режимами всі філософи поїли більше одного разу, ознак starvation у цих тестових запусках не спостерігалось.

`MinEatCount` у режимі 2 вищий (6 проти 5), що в цьому запуску виглядає трохи рівномірніше.

8) Посилання на код

https://github.com/LironeA/DS_Lab4

9) Висновки

У роботі розглянуто і реалізовано два способи синхронізації для задачі Dining Philosophers при `N=5`:

1. лічильний семафор `room=N-1` + синхронізація виделок;
2. глобальний м'ютекс + 5 семафорів виделок.

Обидва підходи орієнтовані на усунення deadlock шляхом руйнування циклічного очікування.

Реалізація на C# (.NET) з `Task` і `CancellationToken` забезпечує керовану симуляцію, детальне логування станів та збір статистики `eatCount`.